ЛР3. Пропуски и выбросы — пошаговый гайд

Результат — Jupyter-ноутбук с разделами 2.1–2.8 по методичке. Ниже для каждого раздела: что сделать, готовый код и что написать в выводе. Код рассчитан на рекомендуемый датасет; для своего замените имена столбцов.

Что сдавать

Раздел	Содержание	Минимум
2.1	Описание датасета, типы признаков, подозрительные значения, описательные статистики	все числовые и категориальные
2.2	Распределение пропусков по признакам и объектам, график, гипотеза о механизме	1 график + вывод
2.3	Способ обработки для **каждого** признака с пропусками + обоснование	таблица
2.4	Сравнение ≥ 2 стратегий для ≥ 1 числового и ≥ 1 категориального признака	графики + выводы
2.5	Гипотеза о распределении для ≥ 2 числовых признаков, Q–Q и тесты	2 признака
2.6	Поиск выбросов ≥ 2 методами, таблица границ, сравнение	таблица
2.7	Обработка выбросов и её последствия	до/после
2.8	Итоговый датасет и сводка изменений	таблица

0. Подготовка среды (один раз на все ЛР3–ЛР6)

0.1. Установка

1. Откройте терминал: **Win + R** → `cmd` → Enter (или терминал в VS Code: **Терминал** → **Создать терминал**).
2. Установите библиотеки (в вашем Python сейчас нет scikit-learn, seaborn и Jupyter):

```
pip install jupyter scikit-learn seaborn scipy statsmodels category_encoders marimo
```

0.2. Создание ноутбука — вариант A: VS Code

1. **Файл** → **Открыть папку** → выберите папку лабораторной (например, `Лаба3`).
2. **Ctrl+Shift+P** → наберите `Create: New Jupyter Notebook` → Enter.
3. Справа вверху **Select Kernel** → **Python Environments** → ваш Python 3.
4. **Ctrl+S** → имя `ЛР3_Фролов.ipynb`.
5. Кнопки над ячейкой: **+ Code** — ячейка кода, **+ Markdown** — текстовая ячейка. Запуск ячейки — **Shift+Enter**.

0.3. Вариант B: Jupyter в браузере

1. В терминале перейдите в папку лабы: `cd C:\Projects\Sem5\ОАД\Лаба3` → `jupyter notebook`.
2. В браузере: **New** → **Python 3 (ipykernel)**.

3. Переименовать: щелчок по «Untitled» вверху.
4. Тип ячейки: **Esc**, затем **M** (Markdown) или **Y** (код). Запуск — **Shift+Enter**.

0.4. Формулы и оформление

В Markdown-ячейке формулы пишутся в TeX: строчная `$x^2$`, выносная `$$\nabla Q = \frac{2}{\ell}X^T(Xw-y)$$`. Заголовки разделов (`## 2.1 Первичный анализ`) делайте **точно по структуре методички**: преподаватель проверяет по ней.

0.5. Перед сдачей

Kernel → Restart Kernel and Run All (в VS Code — **Restart**, затем **Run All**). Все ячейки должны выполниться сверху вниз без ошибок. Экспорт: VS Code → **…** над ноутбуком → **Export** → HTML; Jupyter → **File** → **Download as** → **HTML**.

0.6. Лекционные marimo-ноутбуки

Ноутбуки из `Лекции/…` и `Лабы/…` — это marimo (`.py`). Они читают `data/…` и `img/…` относительно **текущей папки**, поэтому запускайте их из `Лекции/Предобработка`:

```
cd C:\Projects\Sem5\ОАД\Лекции\Предобработка
marimo edit notebooks/missing.py
```

Если marimo всё равно пишет `FileNotFoundError`, скопируйте папки `data` и `img` внутрь `notebooks` — тогда путь найдётся при любом способе запуска. Копии ноутбуков в `Лабы/ЛР3` и `Лабы/ЛР4` без папок `data` и `img` рядом не запустятся.

1. Датасет

Рекомендуемый: Melbourne Housing Snapshot — `https://www.kaggle.com/datasets/dansbecker/melbourne-housing-snapshot`, файл `melb_data.csv` (≈ 13 580 продаж домов, 21 признак). Почему он подходит: в нём есть настоящие пропуски (`Car`, `BuildingArea`, `YearBuilt`, `CouncilArea`), выбросы (`Landsize`, `BuildingArea`, `Price`), подозрительные значения (нулевая площадь, год постройки 1196) и категории. Тот же датасет подойдёт для ЛР4 (EDA), а также для ЛР5–ЛР6 (больше 10 000 строк, цель — `Price`). Скачайте CSV и положите рядом с ноутбуком.

> **Перед началом**
> Числа, которые встречаются ниже («≈ 47 % пропусков» и т. п.), — ориентиры. В выводы пишите **то, что получилось у вас**.

2.1. Первичный анализ

Ячейка Markdown: предметная область (продажи жилья в Мельбурне, 2016–2017), число объектов и признаков, назначение основных признаков, цель — `Price`.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from scipy import stats

pd.set_option('display.max_columns', 50)
df_raw = pd.read_csv('melb_data.csv')
df = df_raw.copy()
print(df.shape)
df.head()
```

```
df.info()          # типы pandas и число непустых значений
df.nunique().sort_values()
```

Типы признаков нужно определить **по смыслу**, а не по dtype:

Тип	Признаки (пример)
Количественные непрерывные	`Price`, `Distance`, `Landsize`, `BuildingArea`, `Lattitude`, `Longtitude`
Количественные дискретные	`Rooms`, `Bedroom2`, `Bathroom`, `Car`, `Propertycount`
Порядковые / временные	`YearBuilt`, `Date`
Номинальные	`Type`, `Method`, `Regionname`, `CouncilArea`, `Suburb`, `SellerG`
Идентификаторы (не признаки)	`Address`, `Postcode` (числом, но это категория)

Подозрительные значения — формально не NaN, но по смыслу ошибка:

```
checks = {
    'BuildingArea == 0': (df['BuildingArea'] == 0).sum(),
    'Landsize == 0':     (df['Landsize'] == 0).sum(),
    'YearBuilt < 1800':  (df['YearBuilt'] < 1800).sum(),
    'Bedroom2 == 0':     (df['Bedroom2'] == 0).sum(),
}
print(checks)
df.loc[df['YearBuilt'] < 1800, ['Address', 'YearBuilt']]
```

Решение: нулевая площадь дома и год постройки до 1800 — это ошибки ввода. Превращаем их в пропуски, чтобы обработать вместе с остальными. `Landsize == 0` для квартир (`Type == 'u'`) правдоподобно — у квартиры нет своего участка, поэтому его не трогаем.

```
df.loc[df['BuildingArea'] == 0, 'BuildingArea'] = np.nan
df.loc[df['YearBuilt'] < 1800, 'YearBuilt'] = np.nan
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
```

Описательные статистики:

```
num_cols = ['Price', 'Rooms', 'Distance', 'Bedroom2', 'Bathroom', 'Car',
            'Landsize', 'BuildingArea', 'YearBuilt']
desc = df[num_cols].describe().T
desc = desc.rename(columns={'50%': 'median', '25%': 'Q1', '75%': 'Q3'})
desc[['min', 'max', 'mean', 'median', 'std', 'Q1', 'Q3']].round(2)
```

```
cat_cols = ['Type', 'Method', 'Regionname', 'CouncilArea']
for c in cat_cols:
    vc = df[c].value_counts(dropna=False)
    print(f'{c}: уникальных {df[c].nunique()}')
    print((vc.head(5) / len(df)).round(3), '\n')
```

Вывод 2.1: перечислите, где `mean` ≫ `median` (признак скошен вправо: `Price`, `Landsize`), где `max` подозрительно велик, какие категории доминируют.

2.2. Анализ пропусков

```
miss = df.isna().mean().sort_values(ascending=False)
miss = miss[miss > 0]
ax = (miss * 100).plot.barh(figsize=(7, 3))
ax.set_xlabel('% пропусков'); ax.invert_yaxis(); plt.show()
miss.round(4)
```

По объектам — сколько пропусков в строке:

```
df.isna().sum(axis=1).value_counts().sort_index()
```

Карта пропусков (видно, совпадают ли они у разных признаков):

```
plt.figure(figsize=(8, 3))
sns.heatmap(df[miss.index].isna().T, cbar=False)
plt.xlabel('объекты'); plt.show()
```

Случайны ли пропуски? Сравните другие признаки у строк с пропуском и без:

```
for c in ['BuildingArea', 'YearBuilt', 'CouncilArea']:
    flag = df[c].isna()
    print(c)
    print(df.groupby(flag)[['Price', 'Distance', 'Rooms']].median(), '\n')
    print(pd.crosstab(df['Type'], flag, normalize='index').round(3), '\n')
```

Формальная проверка для числового признака — тест Манна–Уитни (`Price` у строк с пропуском и без), для категориального — χ^2:

```
flag = df['BuildingArea'].isna()
print(stats.mannwhitneyu(df.loc[flag, 'Price'], df.loc[~flag, 'Price']))
print(stats.chi2_contingency(pd.crosstab(df['Type'], flag))[:2])
```

Вывод 2.2: если распределения заметно отличаются ($p < 0{,}05$), то пропуски **не MCAR**. Вероятно, это MAR: пропуск зависит от наблюдаемых признаков — например, от типа жилья или от того, кто вносил данные. `CouncilArea` по смыслу однозначно определяется пригородом `Suburb`, поэтому восстанавливается почти без ошибки.

2.3. Выбор способов обработки

Ячейка Markdown с таблицей (пример):

Признак	% пропусков	Способ	Почему	Возможные искажения
`Car`	≈ 0,5 %	медиана	пропусков мало, дискретный признак	почти нет
`BuildingArea`	≈ 47 %	медиана по группе (`Type`, `Rooms`) + флаг пропуска	площадь сильно зависит от числа комнат и типа	занижение дисперсии внутри групп
`YearBuilt`	≈ 40 %	медиана по `Suburb` (иначе общая) + флаг	застройка района однородна по годам	«пики» на медианах
`CouncilArea`	≈ 10 %	мода по `Suburb`, остаток — `'Unknown'`	пригород → муниципалитет почти однозначно	минимальные

Удаление строк здесь не подходит: при 47 % пропусков в `BuildingArea` потеряли бы половину данных и сместили выборку.

2.4. Сравнение способов заполнения

Числовой признак — `BuildingArea`

```
from sklearn.preprocessing import StandardScaler
from sklearn.impute import KNNImputer
from sklearn.experimental import enable_iterative_imputer  # noqa: F401
from sklearn.impute import IterativeImputer

col = 'BuildingArea'
base = df[col]

group_med = df.groupby(['Type', 'Rooms'])[col].transform('median')
variants = {
    'без пропусков': base.dropna(),
    'среднее':       base.fillna(base.mean()),
    'медиана':       base.fillna(base.median()),
    'медиана группы': base.fillna(group_med).fillna(base.median()),
}

# KNN и MICE используют другие признаки; KNN требует масштабирования
aux = ['Rooms', 'Bathroom', 'Distance', 'Landsize', col]
sc = StandardScaler()
Z = sc.fit_transform(df[aux])                       # NaN игнорируются при fit
Z_knn = KNNImputer(n_neighbors=5).fit_transform(Z)
variants['KNN (k=5)'] = pd.Series(sc.inverse_transform(Z_knn)[:, -1], index=df.index)
```

```
X_mice = IterativeImputer(random_state=0, max_iter=10).fit_transform(df[aux])
variants['MICE'] = pd.Series(X_mice[:, -1], index=df.index)

pd.DataFrame({k: v.describe() for k, v in variants.items()}).T.round(1)
```

```
fig, ax = plt.subplots(figsize=(8, 4))
for k, v in variants.items():
    sns.kdeplot(v.clip(upper=v.quantile(0.99)), ax=ax, label=k)
ax.legend(); ax.set_title('BuildingArea: плотность до и после заполнения'); plt.show()
```

Вывод: заполнение средним или медианой создаёт высокий пик в одной точке и **уменьшает стандартное отклонение** (сравните `std` с вариантом «без пропусков»). Групповая медиана, KNN и MICE сохраняют форму распределения лучше. Выберите один способ и обоснуйте выбор.

Категориальный признак — `CouncilArea`

```
col = 'CouncilArea'
mode_by_suburb = df.groupby('Suburb')[col].agg(
    lambda s: s.mode().iloc[0] if s.notna().any() else np.nan)
cat_variants = {
    'исходный':          df[col],
    'мода':              df[col].fillna(df[col].mode().iloc[0]),
    "'Unknown'":         df[col].fillna('Unknown'),
    'мода по Suburb':    df[col].fillna(df['Suburb'].map(mode_by_suburb)).fillna('Unknown'),
}
freq = pd.DataFrame({k: v.value_counts(normalize=True, dropna=False)
                     for k, v in cat_variants.items()}).fillna(0)
freq.sort_values('исходный', ascending=False).head(8).round(3)
```

```
freq.sort_values('исходный', ascending=False).head(8).plot.bar(figsize=(9, 3.5))
plt.ylabel('доля'); plt.show()
```

Вывод: глобальная мода раздувает долю одной категории. `'Unknown'` честно показывает пропуск, но добавляет «искусственную» категорию. Заполнение модой по пригороду почти не меняет частоты остальных категорий.

Применяем выбранные способы к рабочему датафрейму (флаги пропуска сохраняем как признаки):

```
for c in ['BuildingArea', 'YearBuilt']:
    df[c + '_was_na'] = df[c].isna().astype(int)
df['Car'] = df['Car'].fillna(df['Car'].median())
df['BuildingArea'] = variants['медиана группы']
yb_sub = df.groupby('Suburb')['YearBuilt'].transform('median')
df['YearBuilt'] = df['YearBuilt'].fillna(yb_sub).fillna(df['YearBuilt'].median())
df['CouncilArea'] = cat_variants['мода по Suburb']
df.isna().sum()[lambda s: s > 0]
```

2.5. Распределения числовых признаков

Берём `Price` и `Landsize` (экстремальные значения есть у обоих). Гипотеза: оба **логнормальны** — положительные, сильно скошены вправо. Проверка: $\ln X$ должен быть близок к нормальному.

```
def dist_report(x, name):
    x = x.dropna()
    x = x[x > 0]
    fig, axes = plt.subplots(1, 3, figsize=(12, 3.2))
    sns.histplot(x, ax=axes[0], bins=60); axes[0].set_title(name)
    stats.probplot(x, dist='norm', plot=axes[1]); axes[1].set_title('Q-Q: X против N')
    stats.probplot(np.log(x), dist='norm', plot=axes[2]); axes[2].set_title('Q-Q: ln X против N')
    plt.tight_layout(); plt.show()
    lx = np.log(x)
    print(f'{name}: skew={stats.skew(x):.2f}, kurt={stats.kurtosis(x):.2f} | '
          f'ln: skew={stats.skew(lx):.2f}, kurt={stats.kurtosis(lx):.2f}')
    print('D\'Agostino для ln X:', stats.normaltest(lx))

dist_report(df['Price'], 'Price')
dist_report(df['Landsize'], 'Landsize')
```

Как читать Q–Q график: точки на прямой — квантили совпадают с теоретическими. Изгиб вверх справа — тяжёлый правый хвост. Для `Price` после логарифмирования точки почти легли на прямую — гипотеза о логнормальности согласуется с данными. При больших выборках тест почти всегда даёт $p < 0{,}05$ даже при малых отклонениях, поэтому главное — форма Q–Q и близость асимметрии к нулю.

2.6. Поиск выбросов

```
def bounds(x, method):
    if method == '3σ':
        m, s = x.mean(), x.std()
        return m - 3 * s, m + 3 * s
    if method == '3σ на ln':
        lx = np.log(x[x > 0]); m, s = lx.mean(), lx.std()
        return np.exp(m - 3 * s), np.exp(m + 3 * s)
    q1, q3 = x.quantile([0.25, 0.75]); iqr = q3 - q1
    if method == 'IQR, K=1.5':
        return q1 - 1.5 * iqr, q3 + 1.5 * iqr
    if method == 'IQR, K=3':
        return q1 - 3 * iqr, q3 + 3 * iqr
    if method == 'квантили 5–95%':
        return x.quantile(0.05), x.quantile(0.95)

methods = ['3σ', '3σ на ln', 'IQR, K=1.5', 'IQR, K=3', 'квантили 5–95%']
rows, masks = [], {}
for col in ['Price', 'Landsize']:
```

```
    x = df[col]
    for m in methods:
        lo, hi = bounds(x, m)
        mask = (x < lo) | (x > hi)
        masks[(col, m)] = mask
        rows.append([col, m, round(lo, 1), round(hi, 1), int(mask.sum()), round(mask.mean() * 100,
2)])
out_table = pd.DataFrame(rows, columns=['признак', 'метод', 'нижняя', 'верхняя', 'число', '%'])
out_table
```

Сравнение множеств выбросов — коэффициент Жаккара $J(A,B) = |A \cap B| / |A \cup B|$:

```
def jaccard(a, b):
    return (a & b).sum() / max((a | b).sum(), 1)
pd.DataFrame([[jaccard(masks[('Price', a)], masks[('Price', b)]) for b in methods] for a in
methods],
             index=methods, columns=methods).round(2)
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3))
sns.boxplot(x=df['Price'], ax=axes[0]); sns.boxplot(x=np.log(df['Price']), ax=axes[1])
axes[1].set_title('ln Price'); plt.show()
```

Вывод 2.6:

- для скошенных признаков правило 3σ даёт **отрицательную** нижнюю границу (бессмысленно) и «не видит» выбросы снизу; среднее и σ сами раздуты хвостом;
- IQR устойчивее: квартили не зависят от хвоста;
- квантильный подход всегда отсекает ровно $\approx 10\,\%$ — это не выбросы, а просто «края»;
- на логарифмической шкале 3σ работает корректно.

Дорогие дома (`Price` > 5 млн) — реальные сделки, а не ошибки. `Landsize` в десятки тысяч м^2 для городского дома — вероятно, ошибка ввода (участок указан для всего комплекса).

2.7. Обработка выбросов

Пример решения: `Price` **не удаляем**, а логарифмируем при моделировании. `Landsize` ограничиваем (винзоризация) по верхней границе IQR с $K = 3$:

```
lo, hi = bounds(df['Landsize'], 'IQR, K=3')
before = df['Landsize'].describe()
df['Landsize'] = df['Landsize'].clip(lower=0, upper=hi)
pd.DataFrame({'до': before, 'после': df['Landsize'].describe()}).round(1)
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3))
sns.histplot(df_raw['Landsize'].clip(upper=df_raw['Landsize'].quantile(0.99)), ax=axes[0], bins=50)
axes[0].set_title('до (обрезано по 99% для показа)')
sns.histplot(df['Landsize'], ax=axes[1], bins=50); axes[1].set_title('после винзоризации')
plt.show()
```

Вывод: среднее и стандартное отклонение заметно уменьшились, медиана и квартили не изменились. Объём данных сохранён, но в точке `hi` появился «пик» — информация о реальной величине больших участков потеряна.

2.8. Сравнение исходного и итогового датасета

```
summary = pd.DataFrame({
    'пропусков до':     df_raw[num_cols + cat_cols].isna().sum(),
    'пропусков после':  df[num_cols + cat_cols].isna().sum(),
    'среднее до':       df_raw[num_cols].mean(),
    'среднее после':    df[num_cols].mean(),
    'std до':           df_raw[num_cols].std(),
    'std после':        df[num_cols].std(),
}).round(2)
print('Строк до:', len(df_raw), 'после:', len(df))
summary
```

```
df.to_csv('melb_clean.csv', index=False)   # пригодится в ЛР4
```

Итоговый вывод: сколько значений заполнено и каким способом; как изменились средние и разбросы; сколько значений ограничено; почему строки не удалялись (или сколько удалено и почему).

Чек-лист

- [ ] Все разделы 2.1–2.8 с заголовками как в методичке.
- [ ] Подозрительные значения найдены и превращены в NaN с обоснованием.
- [ ] График долей пропусков + гипотеза MCAR/MAR/MNAR с проверкой.
- [ ] Таблица способов обработки для **каждого** признака с пропусками.
- [ ] Сравнение ≥ 2 стратегий: числовой (describe + KDE) и категориальный (частоты).
- [ ] 2 признака: гипотеза о распределении, Q–Q, асимметрия, тест.
- [ ] Таблица выбросов: метод, границы, число, доля; сравнение методов.
- [ ] Обработка выбросов + до/после.
- [ ] Сводная таблица изменений; **Restart & Run All** без ошибок.